VM Sizing Report: Delivery Platform Control Server

Date: 2026-09-03 **Branch:** `feat/deployment` at `57f5a23` **Question:** Can the whole stack (Kafka, authoritative, command, web, Caddy) run on a 4 GB GCE VM, and how fast does Kafka eat the disk?

Summary

Every number below was measured by building and running the real stack, not estimated.

Measurement	Result
Cold image build, peak memory above idle	**~0.85 GB** (web is the biggest)
Full stack running, steady state above idle	**~0.85 GB**
Kafka JVM under load	0.4 GB now, capped at **1 GB heap** by default
Kafka disk cost per telemetry record	**~138 bytes** on disk for a 128-byte payload
Worst case total RAM (build during runtime, Kafka heap full)	**~3.2 GB**

Verdict: a 4 GB machine (e2-medium) runs this stack including on-VM builds, with roughly 1 GB to spare. That is a smaller margin than ideal but it works. The earlier assumption that a Next.js build needs 2 to 3 GB was wrong for this app: it has 30 source files and builds in 33 seconds at under 1 GB.

Recommendation: start on **e2-medium (4 GB)** at about $25/month if budget matters, with the config changes in the Recommendations section applied first. Move to e2-standard-2 (8 GB) when a pathing service lands, or if you want to skip the config work. Either way, the boot disk must be larger than the 10 GB default.

Section 6 estimates the services that do not exist yet (pathing, a fuller control service, an operator panel, and teleop). Short version: none of them move the RAM needle much except a grid-based planner, but **teleop video egress can cost more per month than the VM itself.**

1. Build memory

Each image was built one at a time with `--no-cache` so the numbers reflect a fresh VM, not a warm cache. Memory was sampled once a second inside the Docker VM. "Used" means `MemTotal - MemAvailable`, which excludes reclaimable page cache and is what the OOM killer sees.

Idle Docker VM baseline: **736 MB**. Run-to-run variance is about 5%.

Image	Peak used	Above idle	Build time	What it does
web (Next.js)	1,599 MB	+863 MB	51 s	`npm ci` 8.7 s, `next build` 33.4 s
authoritative (Go)	1,228 MB	+492 MB	19 s	Go compile with CGO (librdkafka)
command (Go)	1,051 MB	+315 MB	8 s	Go compile

The compose deploy builds these sequentially, so the peak is the web build alone, not the sum.

Test machine had 10 CPUs. On a 2 vCPU VM the build will be slower (expect 2 to 3 minutes for web) but will use the same or less memory, since webpack and SWC parallelism scale with core count.

2. Runtime memory

Full stack started with `docker compose up -d` (dev compose, no Caddy), waited until Kafka and web reported healthy (13 seconds), then settled for 30 seconds.

Container	RSS at idle	Notes
kafka	362 MB	JVM. `-Xmx1G -Xms1G` default, so it grows to ~1.1 GB under sustained load
kafka-ui	294 MB	JVM. Has no host port in prod, so it is burning RAM for nothing
web	40 MB	Next.js standalone, no requests yet. Expect 150 to 300 MB under load
authoritative	5 MB	Go
command	3 MB	Go
Whole VM	**1,600 MB**	+864 MB above idle. Startup peak was 1,754 MB (+1,018 MB)

Caddy was not measured (it needs a real domain) but runs in under 50 MB.

3. Kafka: what “filling up” actually means

Kafka has two things that grow: the JVM heap and the log directory on disk. They behave very differently.

Heap does not grow with data. It is fixed by `KAFKA_HEAP_OPTS`. The cp-kafka image defaults to `-Xmx1G -Xms1G`. RSS starts low because the JVM does not touch all the heap pages until it needs them, but it will climb to roughly 1.1 GB and stop. Beyond that Kafka uses the OS page cache for log segments, which shows up as “cached” memory and is reclaimable, so it does not count against the OOM killer.

Disk grows without bound by default. Confluent’s image ships with `log.retention.hours=168` (7 days) and `log.retention.bytes=-1` (unlimited). Retention only deletes closed segments, and a segment closes at 1 GiB or after `log.roll.hours` (also 168 h). So a low-rate topic can hold up to **14 days** of data before the first byte is deleted.

MEASURED ON-DISK COST

300,000 records of 128 bytes each were produced into a single-partition topic with Kafka’s own `kafka-producer-perf-test`.

Metric	Value
Records	300,000
Payload per record	128 B
Real disk growth (`du`)	39 MB
Bytes per record on disk	**~138 B**
Kafka JVM RSS before → after	381 → 403 MB
Throughput reached	203k records/s

Batching amortizes Kafka's per-batch header, so the overhead is under 10%. A position update (robot ID, 4 doubles, 1 int64) is about 60 bytes as protobuf or 130 as JSON. The 128-byte test size is a fair stand-in for JSON.

DISK PROJECTION

The code does not yet set a publish rate (the matcher ticks at 1 Hz; robot telemetry rate is undecided). Projections for **2 robots** at 138 B/record:

Rate per robot	Records per week	Disk at 7 days	Disk at 14 days (worst case)
1 Hz	1.2 M	0.17 GB	0.33 GB
10 Hz	12.1 M	1.7 GB	3.3 GB
50 Hz	60.5 M	8.4 GB	17 GB
100 Hz	121 M	17 GB	33 GB

At 10 Hz, the default 10 GB boot disk is gone within the first two weeks once you add the OS and images. At 50 Hz it would be gone in days.

DISK BUDGET FOR THE VM

Item	Size
Debian 12 + Docker Engine	~2.5 GB
Images (kafka 1.4, kafka-ui 0.4, web 0.3, authoritative 0.2, caddy 0.05)	~2.5 GB
Docker build cache	Unbounded. The dev machine has 22 GB of it
Kafka log	See table above
Swap file (recommended)	2 GB

50 GB is the right boot disk. 10 GB fails on images and build cache alone.

4. Sizing options

Machine	RAM	Cost/mo	On-VM builds	Room for pathing
e2-small	2 GB	~$13	No	No
e2-medium	4 GB	~$25	Yes, with swap and heap cap	Light graph planner only
e2-standard-2	8 GB	~$50	Yes, comfortably	Yes
e2-standard-4	16 GB	~$100	Yes	Heavy planner or costmaps

Costs are on-demand in us-central1 and exclude the ~$4 disk and ~$4 static IP.

Worst-case math for e2-medium: 736 idle + 864 stack + 700 Kafka heap growth + 863 web build = **3.2 GB** of 4 GB. The idle baseline on a real Debian VM is lower than Docker Desktop's, so this is conservative.

e2-medium is a shared-core burstable type. It will throttle under sustained CPU load, which this stack rarely produces, but a build plus Kafka plus a planner at once will feel it.

5. Recommendations

Apply these regardless of machine size. They are ordered by how much they buy.

1. **Set Kafka retention explicitly** in `deployments/docker-compose.yml`. Without this the disk fills at whatever rate the robots publish.

   ```
   KAFKA_LOG_RETENTION_HOURS: 72
   KAFKA_LOG_RETENTION_BYTES: 1073741824   # 1 GiB per partition
   KAFKA_LOG_SEGMENT_BYTES: 104857600      # 100 MiB, so retention can
   actually delete
   ```

2. **Cap Kafka's heap.** 1 GB is sized for real clusters. For two robots:

   ```
   KAFKA_HEAP_OPTS: "-Xmx512m -Xms512m"
   ```

3. **Remove kafka-ui from prod.** It already has no host ports in `docker-compose.prod.yml`. Add `profiles: [debug]` to it in the base compose so it only starts with `--profile debug`. Saves about 300 MB.
4. **Add 2 GB of swap** on the VM. Builds spill to disk instead of triggering the OOM killer.

   ```
   sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
   sudo mkswap /swapfile && sudo swapon /swapfile
   echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
   ```

5. **Prune after each deploy.** Add to the end of the deploy workflow's ssh step:

   ```
   docker image prune -f && docker builder prune -f --keep-storage 2GB
   ```

6. **Longer term, build in CI** and push to Artifact Registry so the VM only pulls. This removes the build peak entirely and makes e2-medium comfortable rather than adequate.

6. Estimated increases for planned services

Nothing in this section is measured. The repo has no map, video, teleop, or planner code yet, so these are estimates from the message shapes in the proto, the Go WebSocket hub as written, and typical footprints for each kind of component. Each subsection states its assumptions so they can be checked when the code exists.

6.1 CONTROL SERVICE (ROBOTS CONNECT, TAKE ORDERS, STREAM UPDATES)

This is `authoritative` grown up. It already runs the WebSocket hub, the gRPC order handler, and the matcher. Growth comes from per-robot session state, an order state machine, and fanning live updates out to operator browsers.

Assumptions: 2 robots publishing at 10 Hz, up to 200 concurrent operator or student browsers subscribed to live positions, messages around 150 bytes.

Item	Estimate	Basis
Per WebSocket connection	10 to 20 KB	Two goroutine stacks plus the hub’s 1 KB read and write buffers
200 browser connections	~4 MB	
Fanout load	4,000 msgs/s, ~600 KB/s out	2 robots × 10 Hz × 200 viewers
Process RSS	50 to 150 MB	Go runtime, Kafka client, Supabase client, order and robot state
CPU	under 5% of one vCPU	Go handles this fanout rate with room to spare

Increase: +0.1 GB RAM, negligible CPU. Not a sizing factor.

6.2 PATHING SERVICE

The right answer depends entirely on which kind of planner you build. Three tiers, cheapest first.

Tier A: graph planner in Go. A* or Dijkstra over a sidewalk graph derived from OpenStreetMap. The Danforth campus is under 1 km², which at intersection granularity is a few thousand nodes and roughly 10,000 edges. The graph fits in a few MB. A plan takes well under 1 ms. This can be a package inside `authoritative` rather than a separate service.

Tier B: graph planner in Python. Same algorithm, but with osmnx, networkx, and numpy behind a gRPC or HTTP server. Python plus those libraries is 250 to 300 MB before the graph loads, and networkx stores about 1 KB per node. Plans take 5 to 50 ms. Still fine on a shared core.

Tier C: grid costmap planner with live obstacle merging. A server-side occupancy grid over the campus, updated from robot sensors, replanned at 1 Hz per robot. This is the one that changes the machine.

Grid resolution	Cells for 1 km × 1 km	RAM per layer (uint8)	RAM for 3 layers × 2 robots
0.5 m	4 M	4 MB	24 MB
0.2 m	25 M	25 MB	150 MB
0.1 m	100 M	100 MB	600 MB

RAM is manageable. CPU is not. A* over a 25 M cell grid takes 1 to 5 seconds on one core per plan. Replanning both robots once a second saturates two cores continuously and starves Kafka and the WebSocket hub on any e2 machine.

Tier	RAM	CPU	Machine impact
A: Go graph	+0.05 GB	under 1%	None
B: Python graph	+0.4 GB	5 to 20% bursts	None on 8 GB, tight on 4 GB
C: Grid costmap at 0.2 m	+0.3 GB	100 to 200% sustained	Needs dedicated cores: e2-standard-4 or n2-standard-4

Recommendation: keep local obstacle avoidance on the robot, where the sensors are and where latency matters. The server should only compute the global route, which is Tier A or B. That keeps the VM a coordinator instead of a real-time system, and a cloud round trip should never be in a robot's control loop anyway.

6.3 OPERATOR CONTROL PANEL (LIVE MAP)

The web app serves the panel and the live data rides the WebSocket fanout costed in 6.1. The new cost is the web container actually serving traffic instead of sitting idle.

Item	Estimate
Next.js under real load	150 to 300 MB, up from the 40 MB measured idle
CPU	10 to 30% of a vCPU during bursts of page loads
Map tiles	Serve from a tile CDN, not the VM. Self-hosting tiles adds gigabytes of disk and real CPU

Increase: +0.25 GB RAM.

6.4 TELEOP (VIDEO PLUS JOYSTICK)

Joystick is free: 20 to 50 commands a second of about 50 bytes each. The existing UDP relay in `command` handles that today. Note its receive buffer is 1 KB, so it cannot carry video frames without changes.

Video is the expensive part, and the expense is bandwidth, not RAM.

Assumptions: one 720p30 H.264 camera per robot at 3 Mbps, encoded on the robot. Three ways to move it to an operator:

Approach	VM CPU per stream	VM RAM	Notes
Relay only (WebRTC SFU such as Pion, mediasoup, LiveKit, or raw UDP forward)	3 to 8% of a vCPU	50 to 200 MB for the SFU process	Each extra viewer is another copy out. **This is the right design.**
Transcode on the VM (ffmpeg, MJPEG to H.264)	~1 full vCPU	100 to 300 MB	Avoid. One stream eats the machine's spare core
Peer to peer WebRTC with coturn on the VM	under 5%	~50 MB	Campus NAT will usually force TURN relay, so egress ends up the same as the SFU case

Increase: +0.2 GB RAM, +5 to 15% of a vCPU for two streams. Manageable on any option above e2-small.

The real cost is egress. GCP charges roughly $0.08 to $0.12 per GB leaving the VM to the internet. Inbound from the robots is free. A 3 Mbps stream is 1.35 GB per hour per viewer.

Teleop duty cycle	GB per month, 2 robots, 1 viewer each	Egress cost per month
1 hour per day	81 GB	~$8
4 hours per day	324 GB	~$30
8 hours per day	648 GB	~$60

At 8 hours a day with one operator per robot, video egress costs more than the e2-standard-2 it runs on. Every additional viewer multiplies the row. Two mitigations: drop to 480p or 1.5 Mbps for teleop (halves everything), and only stream when an operator has actually taken control rather than continuously.

The e2 network cap (2 Gbps on e2-medium, 4 Gbps on e2-standard-2) is not a constraint. Two streams to ten viewers is 60 Mbps.

6.5 COMBINED ESTIMATE

Runtime RAM, stacking the measured stack from sections 1 and 2 with the estimates above. The build spike (0.86 GB) is not included since it can be moved to CI.

Scenario	Components	Runtime RAM	Sustained CPU	Machine
Today	Measured stack, Kafka heap grown	1.6 GB	~10%	e2-medium
1: Lean	+ control, Go planner, panel, SFU teleop	2.2 GB	~25%	e2-medium with swap, or e2-standard-2 for comfort
2: Typical	+ control, Python planner, panel, SFU teleop	2.6 GB	~35%	e2-standard-2
3: Grid planner	Scenario 2 with a 0.2 m costmap replanning at 1 Hz	2.9 GB	150 to 250%	e2-standard-4 or n2-standard-4, or move the planner off the box

Add 0.86 GB to each RAM row if builds stay on the VM. Kafka heap in every row is the default 1 GB; the 512 MB cap in section 5 takes 0.5 GB off each.

Bottom line: e2-standard-2 covers everything except a server-side grid planner. Budget for video egress separately from the VM, because at real usage it is the larger bill.

7. Reproducing these measurements

Everything above comes from `scripts/measure-resources.sh`. It runs against the local Docker engine, so any developer or agent with the repo and a filled-in `deployments/.env` can rerun it.

```
# from the repo root
./scripts/measure-resources.sh              # all three experiments, ~3
minutes
./scripts/measure-resources.sh build        # just the cold-build peaks
./scripts/measure-resources.sh run          # just steady-state runtime
./scripts/measure-resources.sh kafka        # just the Kafka disk test
RECORDS=1000000 RECORD_BYTES=64 ./scripts/measure-resources.sh kafka
```

What it does, in order:

1. **Baseline.** Tears the stack down, then reads `/proc/meminfo` inside the Docker VM (via a privileged `nsenter` container) to get idle “used” memory.
2. **build.** Starts a one-per-second memory sampler, then runs `docker compose build --no-cache` for web, authoritative, and command one at a time. Reports the peak used memory in each window.
3. **run.** Runs `docker compose up -d`, polls the kafka and web healthchecks, settles 30 seconds, then prints the startup peak, the settled total, and `docker stats` per container.
4. **kafka.** Starts only Kafka, creates a throwaway topic, produces `RECORDS` records of `RECORD_BYTES` each with `kafka-producer-perf-test`, and diffs `du` on the data directory to get bytes per record. Prints a 7-day projection table for 2 robots.
5. Deletes the test topic, brings the stack down. Raw samples and per-service build logs are left in `.measure/` with a `summary.txt` of key=value pairs.

Requirements: Docker with compose v2, `deployments/.env` populated (the web build needs `SUPABASE_URL`). Works on Docker Desktop (macOS, Windows) and on a Linux host.

Caveats when reading the output:

- Docker Desktop’s idle baseline includes its own VM overhead. A Debian GCE VM idles lower, so absolute totals here are conservative.
- The sampler polls once a second, so sub-second spikes can be missed. Add 10% if you want a safety margin.
- Build memory is roughly independent of CPU count. Build time is not. Expect 3 to 5x longer on 2 vCPUs.

- The Kafka test uses random payloads, which do not compress. Real JSON telemetry would compress well if you enable `compression.type=lz4` on the producer, cutting disk by 3 to 5x.
- `web` is measured idle. Load it before trusting the 40 MB figure for campus-scale traffic.